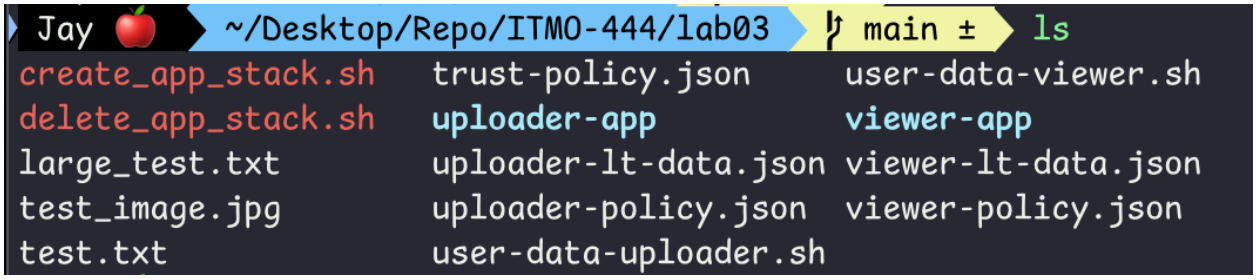
ITMO-444

Two-Tier File Upload & Display with EC2, S3, and IAM Roles

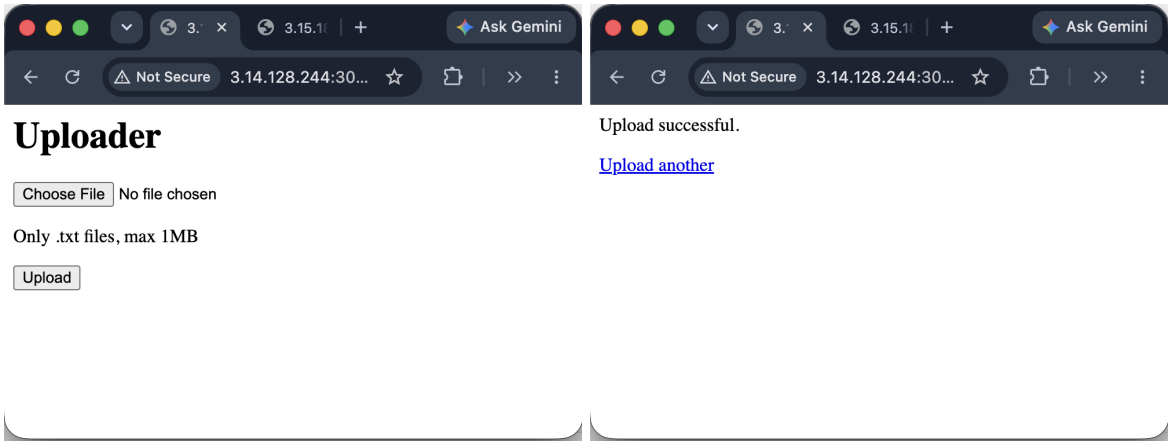
Jay Han
jhan38@hawk.illinoistech.edu

Checklist

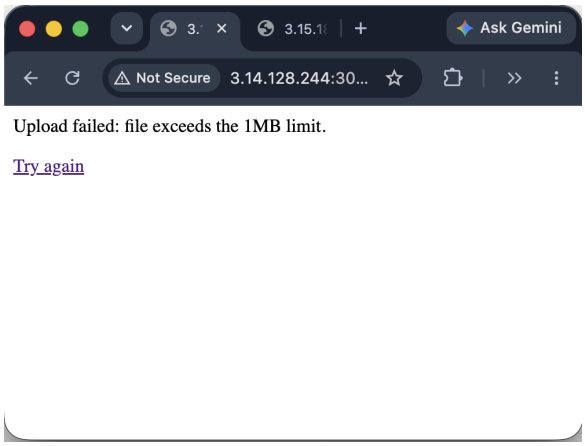
Screenshot of my local lab03/uploader-app/ and lab03/viewer-app/ folders showing app.js and package.json in each



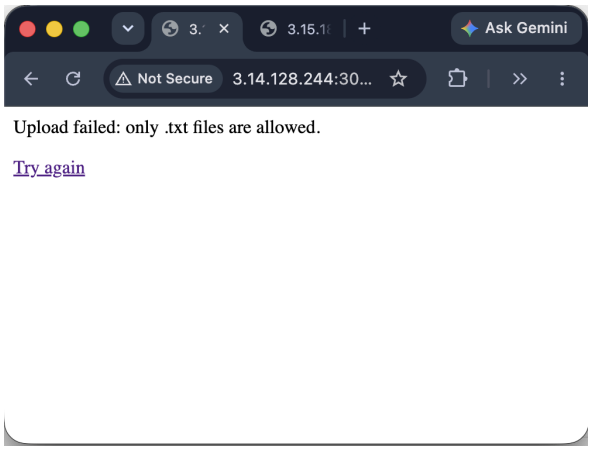
Screenshot of the Uploader page and a successful upload confirmation



Screenshot of attempting to upload a .txt file larger than 1MB, showing the “file exceeds the 1MB limit” error.



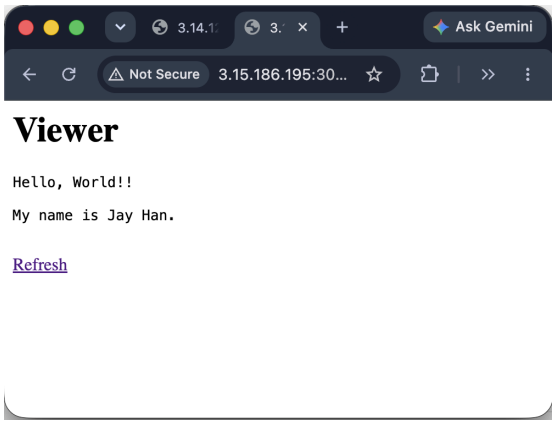
Screenshot of attempting to upload a non-.txt file (e.g. a .jpg or .pdf), showing the “only.txt files are allowed” error.



Screenshot of the Viewer page displaying the uploaded file’s content.



Screenshot of the Viewer page after uploading a second, different file and clicking Refresh, showing the updated content.



Screenshot of aws ec2 describe-launch-templates showing both templates.


```
{
    "LaunchTemplates": [
        {
            "LaunchTemplateId": "lt-0745a391b45a0a7ad",
            "LaunchTemplateName": "itmo-444-544-uploader-lt",
            "CreateTime": "2026-10-02T23:32:23+00:00",
            "CreatedBy": "arn:aws:iam::541195182299:user/Jay_Han",
            "DefaultVersionNumber": 1,
            "LatestVersionNumber": 1,
            "Operator": {
                "Managed": false
            }
        },
        {
            "LaunchTemplateId": "lt-08df43f946bd42900",
            "LaunchTemplateName": "itmo-444-544-viewer-lt",
            "CreateTime": "2026-10-02T23:32:24+00:00",
            "CreatedBy": "arn:aws:iam::541195182299:user/Jay_Han",
            "DefaultVersionNumber": 1,
            "LatestVersionNumber": 1,
            "Operator": {
                "Managed": false
            }
        }
    ]
}
```


Screenshot of aws ec2 describe-launch-template-versions --launch-template-name <uploader-lt-name> after re-running the create script twice, showing more than one version.


```
    {
        "LaunchTemplateId": "lt-0745a391b45a0a7ad",
        "LaunchTemplateName": "itmo-444-544-uploader-lt",
        "VersionNumber": 1,
        "CreateTime": "2026-10-02T23:32:23+00:00",
        "CreatedBy": "arn:aws:iam::541195182299:user/Jay_Han",
        "DefaultVersion": true,
        "LaunchTemplateData": {
            "IamInstanceProfile": {
                "Name": "itmo-444-544-uploader-profile"
            },
            "ImageId": "ami-0a1ff75d5f5ce322c",
            "InstanceType": "t3.micro",
            "KeyName": "ITMO-444-544-ITMO-444-544-lab-keypair",
            "UserData": "IyEvYmluL2Jhc2gKZG5mIHVwZGF0ZSAteQpkbmYgaW5zdGFs
```

```
{
    "LaunchTemplateId": "lt-0745a391b45a0a7ad",
    "LaunchTemplateName": "itmo-444-544-uploader-lt",
    "VersionNumber": 2,
    "CreateTime": "2026-10-02T23:44:16+00:00",
    "CreatedBy": "arn:aws:iam::541195182299:user/Jay_Han",
    "DefaultVersion": false,
    "LaunchTemplateData": {
        "IamInstanceProfile": {
            "Name": "itmo-444-544-uploader-profile"
        },
        "ImageId": "ami-0a1ff75d5f5ce322c",
        "InstanceType": "t3.micro",
        "KeyName": "ITMO-444-544-ITMO-444-544-lab-keypair",
        "UserData": "IyEvYmluL2Jhc2gKZG5mIHVwZGF0ZSAteQpkbmYgaW5zdG
```

aws iam get-role-policy output (or screenshot) for both roles, showing each has only its one intended action on the exact shared.txt key.

```
Jay 🍎 ~/Desktop/Repo/ITMO-444/lab03 ᚠ main ± aws iam get-role-policy --role-name itmo-444-544-uploader-role --policy-name itmo-444-544-uploader-role-policy
{
    "RoleName": "itmo-444-544-uploader-role",
    "PolicyName": "itmo-444-544-uploader-role-policy",
    "PolicyDocument": {
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Action": [
                    "s3:PutObject"
                ],
                "Resource": "arn:aws:s3:::itmo-444-544-jhan38-fileshare/shared.txt"
            }
        ]
    }
}
```

```
Jay 🍎 ~/Desktop/Repo/ITMO-444/lab03 ᚠ main ± aws iam get-role-policy --role-name itmo-444-544-viewer-role --policy-name itmo-444-544-viewer-role-policy
{
    "RoleName": "itmo-444-544-viewer-role",
    "PolicyName": "itmo-444-544-viewer-role-policy",
    "PolicyDocument": {
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Action": [
                    "s3:GetObject"
                ],
                "Resource": "arn:aws:s3:::itmo-444-544-jhan38-fileshare/shared.txt"
            },
            {
                "Effect": "Allow",
                "Action": [
                    "s3:ListBucket"
                ],
                "Resource": "arn:aws:s3:::itmo-444-544-jhan38-fileshare"
            }
        ]
    }
}
```

Screenshot of create_app_stack.sh output showing both instances running with public IPs.

```
-----------------------------------
|        DescribeInstances        |
+---------------+-----------------+
|  uploader-app |  3.14.128.244   |
|  viewer-app   |  3.15.186.195   |
+---------------+-----------------+
```

Screenshot of delete_app_stack.sh completing with no errors.

```
== Removing IAM instance profiles and roles ==
== Emptying and deleting S3 bucket ==
delete: s3://itmo-444-544-jhan38-fileshare/shared.txt
Cleanup complete.
```

One paragraph explaining why the uploader and viewer use separate IAM roles instead of one shared role with both permissions.

In order to implement the Principle of Least Privilege, the uploader and viewer applications utilize distinct IAM roles instead of a single one. This is because of the importance of keeping permissions separate: the uploader EC2 instance gets access only for writing to `shared.txt` (`s3:PutObject`), while the viewer EC2 instance is restricted to reading and checking which objects are in the bucket (`s3:GetObject` and `s3:ListBucket`). If they shared the same role with the power of reading and writing, compromising the public-facing viewer service might lead to an attacker replacing S3 files, whereas compromising the uploader might provide the attacker

with unauthorized access for reading. Isolating the identities of applications limits the functions that the role would be capable of performing in case of a severe security breach.

One or two sentences explaining why embedding code directly into user-data means the instance never needs a Git or S3 credential just to fetch its own code.
When you embed the application code into the user-data script of the instance, it automatically gets embedded in the EC2 metadata when the instance launches. As a result, the boot scripts write the application directly into the server itself. Therefore, the instance can set up and run the web server by itself while it boots without needing any access credentials.